

Найденные ошибки и исправления

Проверено: 607 скриптов, 1419 тегов, 1859 объектов экрана, 323 привязки к ПЛК и весовому терминалу. Методика и воспроизводимые проверки — в разделе 4.

Исправленные тексты скриптов лежат в `fixes/scripts/` (обязательные, 268 файлов) и `fixes/scripts-optional/` (по согласованию, 122 файла). Порядок применения — `fixes/README.md`. **Ни один файл проекта не изменён:** `project/` и `decoded/` — исходное состояние.

1. Сводка

№	Дефект	Критичность	Затронуто	Исправление
D-01	Аварийный останов включает аспирацию и оба направления транспортёров	● критичная	48 присваиваний	FIX-01
D-02	32 обработчика кнопок не привязаны, 30 кнопок мнемосхемы мертвы	● критичная	11 механизмов	переименование в редакторе
D-03	Кнопка сброса аварий пишет не в тот тег — сброс не действует	● высокая	27 тегов	FIX-12
D-04	В контроле стоповых кнопок транспортёров потеряно условие «механизм в работе»	● высокая	18 блоков	FIX-02
D-05	Ложный аларм «Активна блокировка пуска!» при каждом успешном нажатии	● высокая	194 скрипта	FIX-03
D-06	Повторный пуск в течение 120 с идёт без предпусковой сигнализации	● высокая	ManualStart	OPT-01
D-07	Калибровка завышает время открытия на 1 с (двойной инкремент)	● средняя	65 скриптов	FIX-04
D-08	В калибровке нет тайм-аута — вечная блокировка группы задвижек	● высокая	65 скриптов	OPT-02
D-09	Код состояния 3 «Стоп (М)» недостижим — затирается кодом 4	● средняя	MnemoControlN/C/T	описано, правка на решение
D-10	Расчёт производительности: деление на нуль и «залипание» значения	● средняя	Prod	FIX-10
D-11	Провал уставки подачи: при Capacity >= 100 аларма нет вовсе	● средняя	ErrorCapacitControl	FIX-11
D-12	Задание частоты ЧП не ограничено по диапазону	● средняя	7 полей	рекомендация
D-13	Локальный счётчик отпуска теряет импульсы	● средняя	CalculateLocalCounter	FIX-09
D-14	Кнопка сброса ошибки весов устанавливает ошибку	● средняя	VesErrorOff	FIX-07
D-15	Кнопка сброса локального счётчика не работает (условие всегда ложно)	● средняя	ClearLocalCounterButton	FIX-08
D-16	Защита маршрутов T4BlockControl целиком закомментирована	● высокая	вся защита связей	рекомендация
D-17	Кнопка защитной блокировки включает и тут же снимает её	● средняя	BlockControlButton	FIX-05
D-18	Тест сирены: потерянные begin/end, сирена 1-го этажа не снимается	● средняя	AlarmTest	FIX-06

№	Дефект	Критичность	Затронуто	Исправление
D-19	Три пары тегов привязаны к одному и тому же входу/выходу ПЛК	● высокая	6 тегов	правка привязок
D-20	В сообщениях задвижки T7 №6 указан чужой силос	● низкая	T7CloseScript6	FIX-13
D-21	Аспирация целиком без привязки к ПЛК; N7...N9 и T8...T18 тоже	● критичная	25 механизмов	доработка привязок
D-22	Вход аварии не привязан у норий 2, 3, 5, 7, 8, 9	● высокая	6 норий	доработка привязок
D-23	Тренды, архив и группы доступа не настроены	● средняя	проект	доработка
D-24	Indicators и StartVesIndicators — дубликат кода	● низкая	2 скрипта	рекомендация
D-25	Запись в статусные теги весового терминала вместо команд L1_D1_Cmd	● средняя	3 скрипта	FIX-07 + рекомендация

2. Обязательные исправления

D-01 ● Аварийный останов включает механизмы

AlarmStopButton_OnClick. Нории останавливаются правильно (RelayOutN* := False), а дальше:

```
RelayOutC1.Value := True;           // аспирация 1 – ВКЛЮЧАЕТСЯ
...
RelayOutForwardT1.Value := True;    // транспортёр 1 вперёд – ВКЛЮЧАЕТСЯ
RelayOutReversT1.Value := True;    // и одновременно назад
```

Что True — это именно пуск, подтверждается независимо: ManualStart выдаёт команду пуска как RelayOutX.Value := True, а StopButtonX_OnClick останавливает как RelayOutX.Value := False. То есть нажатие аварийного останова запускает 12 аспираций и подаёт на 18 транспортёров команды на прямое и реверсивное вращение сразу.

FIX-01: 48 присваиваний True → False.

D-02 ● Мёртвые кнопки на мнемосхеме

В Simple-Scada обработчик привязывается к событию объекта по имени <ИмяОбъекта>_<Событие>. Из 361 обработчика события 329 имеют соответствующий объект, а 32 — нет; имена этих скриптов оставлены в заводской нумерации, тогда как объекты пронумерованы сплошь.

Скрипт (не привязан)	Управляет тегами	Свободный объект	Механизм
StartButtonC1a_OnClick, StopButtonC1a_OnClick	C5	StartButtonC5, StopButtonC5	Аспирация 1а
StartButtonC2a_OnClick, StopButtonC2a_OnClick	C6	StartButtonC6, StopButtonC6	Аспирация 2а
StartButtonN1a_OnClick, StopButtonN1a_OnClick	N7	StartButtonN7, StopButtonN7	Нория 1а
StartButtonN2a_OnClick, StopButtonN2a_OnClick	N8	StartButtonN8, StopButtonN8	Нория 2а

Скрипт (не привязан)	Управляет тегами	Свободный объект	Механизм
StartButtonN11_OnClick, StopButtonN11_OnClick	N9	StartButtonN9, StopButtonN9	Нория 11
Start(Forward\ Revers)ButtonT8a_OnClick, StopButtonT8a_OnClick	T12	...ButtonT12	Транспортёр 8а
Start(Forward\ Revers)ButtonT9a_OnClick, StopButtonT9a_OnClick	T14	...ButtonT14	Транспортёр 9а
Start(Forward\ Revers)ButtonT17a_OnClick, StopButtonT17a_OnClick	T15	...ButtonT15	Транспортёр 17а
Start(Forward\ Revers)ButtonT25_OnClick, StopButtonT25_OnClick	T16	...ButtonT16	Транспортёр 25
Start(Forward\ Revers)ButtonT24_OnClick, StopButtonT24_OnClick	T17	...ButtonT13*	Транспортёр 24
Start(Forward\ Revers)ButtonT22_OnClick, StopButtonT22_OnClick	T18	...ButtonT18	Транспортёр 22
T4CalButton6_OnClick	задвижка T4 силос 6-16	T6CalButton16 — опечатка в имени объекта	калибровка
AlarmTest_OnClick	сирены	AlarmTestButton	тест сигнализации
VesStartImage_OnClick, VesStopImage_OnClick	весы	объектов нет	см. D-25

* Пара T13/T17 перекрещена: объект ...ButtonT17 уже привязан к скрипту, который управляет тегами T13 («Транспортёр 17»), а объект ...ButtonT13 свободен. Если объекты нумеровались по индексу тега (как остальные 12 из 18), то нужно переименовать ...ButtonT17_OnClick → ...ButtonT13_OnClick, а ...ButtonT24_OnClick → ...ButtonT17_OnClick. **Перед переименованием сверьте расположение кнопок на мнемосхеме** — из бинарного файла страницы видны имена и геометрия, но не подписи рядом.

На странице есть объекты T4CalButton1...5 и T4CalButton7...10, а вместо T4CalButton6 создан T6CalButton16 — то есть кнопка калибровки задвижки силоса 6-16 названа как кнопка группы T6 с несуществующим номером 16. Достаточно переименовать объект T6CalButton16 → T4CalButton6.

Итог: механизмы C5, C6, N7, N8, N9, T12, T14, T15, T16, T17, T18 нельзя ни пустить, ни остановить с экрана; кнопка теста сирены не работает; ещё 30 кнопок на мнемосхеме не делают ничего.

Исправление только в редакторе (переименование скриптов или объектов), текстовой правкой не лечится. Исключение — AlarmTest_OnClick: в fixes/scripts/ он уже сохранён под правильным именем AlarmTestButton_OnClick (FIX-06).

D-03 ● Сброс аварий пишет не в тот тег

В проекте два похожих семейства тегов:

Семейство	Группа	Кто пишет	Кто читает
StatusAlarmN*, StatusAlarmT*	Таймеры / ... / Таймеры аварий	MnemoControlN, MnemoControlT	MnemoControlN, MnemoControlT
AlarmStatusN*, AlarmStatusT*	Статусы механизмов	AlarmDropButton_OnClick	НИКТО

Мнемосхема проверяет StatusAlarmX, а кнопка «Сброс аварии механизмов» обнуляет AlarmStatusX — 27 тегов, которые никем не читаются. Авария снимается только сама, по исчезновению входа RelayAlarmX.

FIX-12: 27 присваиваний AlarmStatus* → StatusAlarm*.

D-04 ● Потеряно условие в контроле стоповых кнопок

StopButtonsControl, блоки норий и аспирации:

```
if StatusN1.Value = 1 then          // только если механизм в работе (A)
begin
  if RelayStopInN1.AsBool = False then
    begin RelayOutN1.Value := False; StatusStartN1.Value := 16; end;
  end;
```

Блоки транспортёров — те же 18 штук, но без внешнего условия (на строках 219–220 видны подряд идущие комментарии //ТРАНСПОРТЕРЫ начало и //ТРАНСПОРТЕРЫ конец — след неудачной вставки):

```
//Транспортер 1 начало
begin
  if RelayStopInT1.AsBool = False then          // ← здесь должно быть if StatusT1.Value = 1 then
    begin ... StatusStartForwardT1.Value := 16; ... end;
  end;
```

Следствие: пока цепь стоповой кнопки транспортёра не под напряжением, скрипт каждую секунду переписывает StatusStartForwardTn/StatusStartReversTn в 16. Заявка на пуск (код 5) не доживает до 20-й секунды в ManualStart — транспортёр не запустится, и причина в журнале не видна.

FIX-02: во все 18 блоков вставлено if StatusTn.Value = 1 then.

D-05 ● Ложный аларм при каждом нажатии кнопки задвижки

Все 194 обработчика кнопок задвижек построены так:

```
if T6BlockSelectStatus.AsBool = False then
begin
  ...
  T6BlockSelectStatus.Value := True;          // сами ставим блокировку
  AddMessage(... 'Начало открытия задвижки ...');
end;
if T6BlockSelectStatus.AsBool = True then      // видим СВОЮ же запись
  AddMessage(... mkAlarm, 'Активна блокировка пуска! Дождитесь выполнение операции...');
```

Второе условие проверяет флаг, который скрипт сам только что изменил, поэтому аларм выдаётся **всегда** — и при успешном старте тоже. Журнал (и PostgreSQL) забивается ложными авариями, оператор перестаёт им верить.

FIX-03: состояние блокировки снимается в локальную переменную BlockedBefore до изменения флага:

```
var
  BlockedBefore: Boolean;
begin
  BlockedBefore := T6BlockSelectStatus.AsBool;
  if BlockedBefore = False then ...
  if BlockedBefore = True then AddMessage(...);
```

D-07 ● Двойной инкремент времени калибровки

T*CalScriptN, такт 6: выход открытия включается и время открытия инкрементируется в том же блоке, а следующий блок инкрементирует его ещё раз, потому что выход уже стал True:

```

if CalTimer > 5 then if CalTimer < 7 then
begin
  LatchRunOpen.Value := True;
  CalOpenTime.Value := CalOpenTime.Value + 1;  // ← лишний
end;
if CalTimer > 5 then if LatchRunOpen.AsBool = True then
  CalOpenTime.Value := CalOpenTime.Value + 1;

```

Время открытия завышается на 1 с и переносится в уставку тайм-аута 9 + CalOpenTime алгоритма открытия.

FIX-04: лишний инкремент убран в 65 скриптах.

D-10 ... D-15, D-17, D-18, D-20 — остальные обязательные

№	Что было	Что стало
D-10	Capacity := ... * (3600 / (Timer - 1)) / 1000 — при Timer = 1 деление на ноль; после простоя интервал считается от начала простоя, значение «залипает»	FIX-10: интервал 3600 / Timer, база обновляется вместе со сбросом таймера, при простое > 120 с Capacity := 0
D-11	if Capacity > 34 then if Capacity < 100 then аларм; if Capacity < 34 then сброс — при Capacity >= 100 и ровно 34 не выполняется ничего	FIX-11: if Capacity > 34 ... else ...
D-13	Каждый второй такт уходил на перебазирование CalculateCount, приращение за этот такт терялось	FIX-09: база обновляется каждый такт после учёта приращения
D-14	L1_D1_S_Error.Value := 1 — кнопка «сброс ошибки» ставит признак ошибки	FIX-07: := 0. Проверить по протоколу Тензо-М: сброс через статусный тег может быть нештатным, правильнее командой L1_D1_Cmd
D-15	if ClearLocalCounter.AsBool = True then LocalCounter := 0 — тег ClearLocalCounter никем не пишется, условие всегда ложно	FIX-08: счётчик обнуляется безусловно + запись в журнал
D-17	Два if вместо if/else: флаг включается и в следующей же строке выключается, в журнал идут сразу оба сообщения	FIX-05: if ... else ...
D-18	if A then if B then begin ... end else X := True; Y := True; — Y (сирена 1-го этажа) включается всегда, потерян begin/end	FIX-06: условие через and, обе ветви в begin/end; скрипт переименован в AlarmTestButton_OnClick
D-20	T7CloseScript6: «Задвижка Транспортера 6 Силос 46» в двух сообщениях, хотя остальные 5 скриптов этой задвижки говорят «Транспортера 7 Силос 56»	FIX-13

3. По согласованию и рекомендации

D-06 ● Пуск без предпусковой сигнализации (ОПТ-01)

ManualStart держит окно активности 120 с и выдаёт команды с 20-й секунды, а сирену — только первые 10 с. Кнопка пуска ставит ManualStartStatus := True, но таймеры не сбрасывает. Сценарий:

```

t=0   пуск нории 1 → сирена
t=20  нория 1 пущена
t=30  пуск транспортёра 5 → ManualTimerDelay = 30 >= 20
t=31  транспортёр 5 запущен БЕЗ предпусковой сигнализации

```

ОПТ-01 (57 скриптов пуска): вместе с ManualStartStatus := True таймеры обнуляются, то есть каждая новая заявка заново даёт 10 с сирены и 20 с задержки. Побочный эффект: заявка,

поданная в течение 20 с после предыдущей, отодвигает пуск обеих на 20 с от последнего нажатия. Требуется согласования с регламентом объекта.

D-08 ● Калибровка без тайм-аута (OPT-02)

В T*OpenScriptN/T*CloseScriptN тайм-аут есть ($> 9 + \text{CalTime}$), а в T*CalScriptN — нет: единственный выход из алгоритма — срабатывание концевики. Условие $\text{CalTimer} > 5 + \text{CalOpenTime}$ не сработает, потому что CalOpenTime растёт синхронно с CalTimer, пока выход открытия активен. При неисправном концевики скрипт крутится бесконечно, а T*BlockSelectStatus = True блокирует управление **всеми** задвижками группы до перезапуска проекта.

OPT-02 (65 скриптов): предельное время 600 с, по нему выход снимается, блокировка сбрасывается, состояние задвижки — 5 «авария», сообщение в журнал. Уставку 600 с надо согласовать с фактическим ходом задвижек.

D-09 ● Код состояния 3 «Стоп (М)» недостижим

В MnemoControlN/C/T четыре ветви идут подряд, не через else. В ветви (RelayOut=1, RelayIn=0) по истечении 3 с выполняется $\text{RelayOutX} := \text{False}$; $\text{StatusX} := 3$, после чего **в том же проходе** срабатывает ветвь (0, 0) и ставит $\text{StatusX} := 4$. Оператор никогда не видит «Стоп (М)» после неуспешного пуска — только «Стоп (А)».

Автоматически не исправлено: непонятно, какое состояние задумано штатным. Варианты — сделать ветви взаимоисключающими (else if) либо снимать выход в конце блока. Решение за автором проекта.

D-12 ● Задание частоты без ограничения

$\text{T} < \text{FreqTarget}$: $\text{FreqToPLC} := \text{TargerFreq} * 10000 / 50$. Ни в поле окна FreqWindow, ни в скрипте нет проверки диапазона: ввод 100 Гц даёт 20000 в ПЛК (при шкале 0...10000 на 0...50 Гц). Рекомендуется задать в свойствах поля границы 0...50 Гц и добавить в скрипт отсечение.

D-16 ● Защита маршрутов не реализована

T4BlockControl — периодический скрипт, тело целиком закомментировано (94 строки). Задумана проверка: если маршрут T1-T4 или T2-T4 собран (зелёные LineReversT1/LineForwardT1 и LineT1Latch*), а T4 остановлен (ImageT4_1 красный) — остановить T1/T2 и соответствующую норию. Кнопка BlockControlButton и индикатор BlockControlMnemo на экране есть, тег BlockControl переключается, но ни одной блокировки за ним не стоит.

Логика опирается на **цвета объектов мнемосхемы** вместо тегов — это хрупко (зависит от порядка выполнения скриптов отрисовки). При доработке рекомендуется переписать условия через StatusT* и T*LatchStatusByte*.

D-19 ● Один вход/выход ПЛК на два тега

Символ в ПЛК	Теги	Следствие
mo1D032	Alarm16Level, Alarm1Level	обе сирены на одном выходе; сирена 1-го этажа физически не подключена
mi1DI6	RelayInReversT3, RelayInReversT5	ответ реверса T5 берётся с входа T3
t1DI22	MembSensorAT2, MembSensorAT3	датчик подпора T3 повторяет T2

Дополнительно: RelayOutReversT2 и RelayOutReversT3 выходов не имеют, хотя входы реверса у них заведены — вероятно, эти транспортёры однонаправленные и входы реверса лишние. Правится в редакторе тегов.

D-21 ● Незавершённые привязки к ПЛК

Оборудование	Привязано	Не привязано
Нории	N1...N6	N7, N8, N9, N10
Аспирация	—	C1...C14, все
Транспортёры	T1...T7	T8...T20

Ни один тег аспирации (RelayOutC*, RelayInC*, RelayStopInC*) не привязан к ПЛК. Кнопки, мнемосхема, сообщения и все 12 пар обработчиков есть, но команда никуда не уходит, а StatusC* никогда не станет 1 «в работе», потому что RelayInC* всегда False. Аспирация управляется «вслепую».

То же для транспортёров T8...T18: кнопки и обработчики есть (кроме мёртвых по D-02), выходов нет.

Это и есть основной объём доработки проекта: 296 привязок OPC UA сделано, для полного состава нужно ещё около 190 (14 аспираций × 3 + 11 транспортёров × 5 + 4 нории × 4).

D-22 ● Вход аварии не привязан у 6 норий

RelayAlarmN* привязан только у N1, N4, N6. У норий 2, 3, 5, 7, 8, 9 тег читается в MnemoControlN, но всегда False — авария этих норий не детектируется, StatusAlarmN* не поднимется, код состояния 10 «Авария» недостижим.

D-23 ● Тренды, архив и доступ не настроены

- Trends.str — один раздел и пустая «Группа 1»: тренды не настроены, хотя архив в PostgreSQL объявлен.
- ArcSystem.sar — 8 байт, настройки архивации по сути пустые.
- Groups.seg — 8 байт: группы доступа не заданы, то есть разграничения прав операторов нет, и ConfirmMessage(GetClientName) подтверждает сообщения от любого клиента.
- 14 тегов телеметрии ЧП (T<n>Freq, T<n>Power) не используются ни в одном скрипте — проверьте, что они выведены полями экрана, иначе токи и частоты нигде не видны.

D-24 ● Мелкие замечания

- Indicators (периодический) и StartVesIndicators (при запуске) — почти дословный дубликат 60 строк; расходятся только способом чтения (.Value = 1 против .AsBool = True) и обнулением Capacity. Логику стоит держать в одном скрипте.
- В Indicators/StartVesIndicators строки заданы двойными кавычками ("Закрита") — для Pascal это нетипично; проект компилируется, но лучше привести к одинарным.
- Опечатка в именах тегов: T<n>TargerFreq вместо TargetFreq (14 тегов). Переименование сломает совместимость — оставлено как есть.
- 214 периодических скриптов на такт 1 с (из них 195 — алгоритмы задвижек, каждый со своим счётчиком). Нагрузка на сервер растёт линейно с числом задвижек; при дальнейшем расширении логику стоит свернуть в один скрипт с циклом по индексу.

D-25 ● Запись в статусные теги весового терминала

Три обработчика пишут в теги, которые терминал отдаёт только на чтение:

Скрипт	Строка	Что происходит
VesErrorOff_OnClick	L1_D1_S_Error.Value := 1	«сброс ошибки» ставит ошибку (D-14)
VesStartImage_OnClick	L1_D1_S_Start.Value := 1	зажигает лампу «Пуск» в Indicators, команду терминалу не даёт
VesStopImage_OnClick	L1_D1_S_Stop.Value := 1	то же для лампы «Стоп»

Объектов `VesStartImage` и `VesStopImage` на экране нет, поэтому последние два скрипта не выполняются. Штатное управление циклом — через `L1_D1_Cmd` (`VesInWork` = 2, `VesStopCircle` = 3), что и реализовано рабочими кнопками. При доработке эти три скрипта следует либо удалить, либо перевести на `L1_D1_Cmd` согласно протоколу Тензо-М.

4. Как проверено

Все выводы получены статическим анализом распакованного проекта, без запуска SCADA. Инструменты в `tools/`, каждый запускается из корня `scada/bkhp-silkorp`:

Инструмент	Что делает
<code>parse_ssc.py</code>	разбор <code>Scripts.ssc</code> : 607 скриптов, тип запуска, привязки, 99.7 % байт файла
<code>parse_svrr.py</code>	дерево тегов из <code>Variables.svrr</code> : 64 группы, 1419 тегов, 100 % байт
<code>parse_svr.py</code>	привязки к OPC из <code>Variables.svr</code> : 1417 из 1419 записей
<code>check_tags.py</code>	чтения и записи каждого тега по всем скриптам; мёртвые теги, непривязанные входы/выходы
<code>check_index.py</code>	несоответствия индексов в одноптипных скриптах задвижек (найдено 0)
<code>check_selfclobber.py</code>	шаблон «условие проверяет флаг, который скрипт сам изменил» (D-05, D-07, D-09, D-17)
<code>coverage.py</code>	матрица «тег → объект → обработчик → привязка к ПЛК» по всем механизмам
<code>apply_fixes.py</code>	генерация <code>fixes/</code> — каждая правка с проверкой шаблона, при несовпадении падает
<code>gen_docs.py</code>	справочные таблицы <code>docs/reference/</code>

Проверки, давшие отрицательный результат (ошибок не найдено):

- индексы тегов во всех 194 скриптах кнопок и 195 алгоритмах задвижек соответствуют номеру задвижки в имени скрипта;
- подписи силосов и маршрутов согласованы между всеми скриптами одной задвижки — единственное расхождение D-20;
- ни один идентификатор в скриптах не остался неразрешённым (все 1419 тегов, 1859 объектов, встроенные функции); из 5 «неизвестных» имён 4 — регистровые варианты цветовых констант, 1 — локальная переменная;
- формулы всех семи скриптов `T<n>FreqTarget` идентичны и без опечаток;
- распределение ламп задвижек 1–5 → `_1`, 6–10 → `_2` во всех семи группах выполнено верно (проверялось как возможная copy-paste ошибка);
- все привязанные выходы ПЛК пишутся хотя бы одним скриптом.